BÁO CÁO ASSIGNMENT 1
DEVELOPMENT MINIMALIST OF THE LLAMA2 MODEL

Thành viên nhóm:
Nguyễn Thị Ngọc Lan - 23020390
Tạ Giang Thùy Loan - 23020397
Hoàng Ngọc Điệp - 23020357

1. Mục tiêu

Bài thực hành tập trung xây dựng một phiên bản tối giản của Llama2 để hiểu rõ các thành phần chính của mô hình ngôn ngữ dựa trên Transformer. Sau khi hoàn thiện phần hiện thực, nhóm đánh giá mô hình ở ba thiết lập: sinh văn bản, zero-shot prompting và fine-tuning cho phân loại cảm xúc.

Checkpoint sử dụng là stories42M.pt, mô hình 8 lớp với khoảng 42 triệu tham số, được pretrained trên TinyStories. Hai bộ dữ liệu đánh giá là SST-5 và CFIMDB.

2. Thành phần đã thực hiện

File	Hàm / lớp	Nội dung thực hiện
llama.py	Attention.forward	Tính scaled dot-product attention, softmax trên ma trận attention, áp dụng dropout và tổng hợp value.
llama.py	RMSNorm.norm	Chuẩn hóa RMS theo chiều cuối của tensor với epsilon để tăng ổn định số học.
llama.py	Llama.forward	Ghép embedding, các LlamaLayer và projection đầu ra để sinh logits dự đoán token kế tiếp.
llama.py	Llama.generate	Sinh chuỗi bằng greedy decoding khi temperature = 0 và temperature sampling khi temperature > 0.
rope.py	apply_rotary_emb	Áp dụng rotary positional embedding cho query và key.
optimizer.py	AdamW.step	Cập nhật tham số theo AdamW, có bias correction và decoupled weight decay.
classifier.py	LlamaEmbeddingClassifier.forward	Lấy biểu diễn token cuối, áp dụng dropout và tầng tuyến tính để phân loại câu. Thử nhiều loại Pooling

```python
def compute_query_key_value_scores(self,
                                   query: torch.Tensor,
                                   key: torch.Tensor,
                                   value: torch.Tensor) -> torch.Tensor:
    '''
    Jointly compute Scaled Dot Product Attention (see Section 3.2.1 in
    https://arxiv.org/abs/1706.03762 for details). The query, key, and
    value tensors each have shape (bs, n_local_heads, seqlen, head_dim).
    An optimal implemention will jointly computing attention for multiple
    heads (n_local_heads of them) at once using matrix/tensor operations.

    Make sure to use attention_dropout (self.attn_dropout) on the computed
    attention matrix before applying it to the value tensor.
    '''
    # todo
    scores = torch.matmul(query, key.transpose(-2,-1))
    # bs * n_heads * seq_len * seq_len
    scores = scores/math.sqrt(self.head_dim)
    attn = torch.softmax(scores, dim=-1)
    attn = self.attn_dropout(attn)
    output = torch.matmul(attn, value)
    return output
```

Hình 1. Attention.forward: scaled dot-product attention.

```python
def _norm(self, x):
    """
    Compute the root mean square normalization. Use Equation 4 under
    Section 4 of https://arxiv.org/abs/1910.07467 as a reference. Add
    the given epsilon value (self.eps) to the tensor's norm (i.e. inside
    the square root in Equation 4) before normalizing the tensor.

    Args:
        x (torch.Tensor): The input tensor.

    Returns:
        torch.Tensor: The normalized tensor.
    """
    # todo
    mean_square = x.pow(2).mean(dim=-1, keepdim=True)
    rms = torch.rsqrt(mean_square + self.eps)
    return x * rms
```

Hình 2. RMSNorm.norm: chuẩn hóa RMS.

```python
def forward(self, input_ids):
    logits, hidden_states = self.llama(input_ids)

    pooled = hidden_states.mean(dim=1)
    dropped = self.dropout(pooled)

    logits = self.classifier_head(dropped)
    log_probs = F.log_softmax(logits, dim=-1)

    return log_probs
```

Thử với các phiên bản file classifier.py khác nhau

```python
class LlamaLayer(nn.Module):
    def __init__(self, layer_id: int, config: LlamaConfig):
        super().__init__()
        self.n_heads = config.n_heads
        self.dim = config.dim
        self.head_dim = config.dim // config.n_heads
        self.attention = Attention(config)
        self.feed_forward = FeedForward(
            dim=config.dim,
            hidden_dim=config.hidden_dim,
            multiple_of=config.multiple_of,
            dropout=config.dropout,
        )
        self.layer_id = layer_id
        self.attention_norm = RMSNorm(config.dim, eps=config.layer_norm_eps)
        self.ffn_norm = RMSNorm(config.dim, eps=config.layer_norm_eps)

    def forward(self, x):
        '''
        This is the forward pass of the basic transformer building block. This is a
        modernized version of the block shown on the left of Figure 1 on
        https://arxiv.org/pdf/1706.03762.pdf.

        The transformer block should consist of:
        1) layer normalization of the input (via Root Mean Square layer normalization)
        2) self-attention on the layer-normalized input
        3) a residual connection (i.e., add the input to the output of the self-attention)
        3) layer normalization on the output of the self-attention
        4) a feed-forward network on the layer-normalized output of the self-attention
        5) add a residual connection from the unnormalized self-attention output to the
           output of the feed-forward network
        '''
        # todo
        norm_x = self.attention_norm(x)
        attn_out = self.attention(norm_x)
        h = x + attn_out
        norm_h = self.ffn_norm(h)
        ffn_out = self.feed_forward(norm_h)
        out = h + ffn_out
        return out
```

Hình 3. LlamaLayer.forward: attention, FFN và residual connection.

```python
@torch.inference_mode()
def generate(self, idx, max_new_tokens, temperature=1.0):
    """
    Take a conditioning sequence of indices idx (LongTensor of shape (b,t)) and complete
    the sequence max_new_tokens times, feeding the predictions back into the model each time.
    We perform this generation using basic temperature sampling. Note that we are not using
    nucleus sampling (i.e. limiting ourselves to sampling from the top-k most probable tokens
    at each timestep), though this is often used in conjunction with temperature sampling,
    Most likely you'll want to make sure to be in model.eval() mode of operation for this.
    Also note this is a super inefficient version of sampling with no key/value cache.
    """
    for _ in range(max_new_tokens):
        # if the sequence context is growing too long we must crop it at block_size
        idx_cond = idx if idx.size(1) <= self.params.max_seq_len else idx[:, -self.params.max_seq_
        # forward the model to get the logits for the index in the sequence
        logits, _ = self(idx_cond)
        logits = logits[:, -1, :] # crop to just the final time step
        # todo


        if temperature == 0.0:
            # select the single most likely index
            # greedy decoding
            idx_next = torch.argmax(logits, dim=-1, keepdim=True)
        else:
            '''
            Perform temperature sampling:
            1) identify  the logits at the final step.
            2) scale (divide) these probabilities by the given temperature.
            3) normalize the scaled logits with a softmax to obtain scaled probabilities.
            4) sample from the scaled probability distribution.

            Note that we are not using top-k sampling/nucleus sampling in this procedure.
            '''
            logits = logits/temperature
            probs = F.softmax(logits, dim=-1)

            idx_next = torch.multinomial(probs, num_samples=1)
        # append sampled index to the running sequence and continue
        idx = torch.cat((idx, idx_next), dim=1)


    return idx
```

Hình 4. Llama.generate: greedy decoding và temperature sampling.

3. Kiểm thử và sinh văn bản

Ba kiểm thử bắt buộc đều chạy thành công: optimizer_test.py, rope_test.py và sanity_check.py. Điều này xác nhận các thành phần AdamW, RoPE và forward pass của Llama khớp với yêu cầu kiểm thử.

Ở thiết lập sinh văn bản, mô hình tạo hai chuỗi tiếp nối từ cùng một prompt với temperature 0.0 và 1.0. Kết quả cho thấy mô hình sinh được tiếng Anh có cấu trúc ngữ pháp, dù nội dung chưa hoàn toàn phù hợp ngữ cảnh phim do checkpoint được pretrained trên TinyStories.

```
In [4]: !python optimizer_test.py

Optimizer test passed!

In [5]: !python rope_test.py

Rotary embedding test passed!

In [6]: !python sanity_check.py

Your Llama implementation is correct!
```

Hình 1. Kết quả chạy optimizer_test.py, rope_test.py và sanity_check.py.

```
In [7]: !python  run_llama.py  --option  generate

args: {'train': 'data/cfimdb-train.txt', 'dev': 'data/cfimdb-dev.txt', 'test': 'data/cfimdb-test.txt',
'label_names': 'data/cfimdb-label-mapping.json', 'pretrained_model_path': 'stories42M.pt', 'max_senten
ce_len': None, 'seed': 1337, 'epochs': 5, 'option': 'generate', 'use_gpu': False, 'generated_sentence_
low_temp_out': 'generated-sentence-temp-0.txt', 'generated_sentence_high_temp_out': 'generated-sentenc
e-temp-1.txt', 'dev_out': 'cfimdb-dev-prompting-output.txt', 'test_out': 'cfimdb-test-prompting-outpu
t.txt', 'batch_size': 8, 'hidden_dropout_prob': 0.3, 'lr': 2e-05}
load model from stories42M.pt
Temperature is 0.0
I have wanted to see this thriller for a while, and it didn't disappoint. Keanu Reeves, playing the he
ro John Wick, is this day. He was playing with his toy car, driving it around the living room. Suddenl
y, he heard a loud crash. He had broken the car and was very sad.
John was angry and he shouted at his little brother. He was only three years old and he was only thre
e. He was only three years old. He was very ups
---------------
Wrote generated sentence to generated-sentence-temp-0.txt.
load model from stories42M.pt
Temperature is 1.0
I have wanted to see this thriller for a while, and it didn't disappoint. Keanu Reeves, playing the he
ro John Wick, is coming a new friend to play with."
Iwriting brought a mask, but I wore plain white beats. The sun shone brightly. I be a happy, obedient
lung.
It'scious to be a true hero. What should I play?"here Ialen Once upon a time, there was a jolly youth
named Lucy.
---------------
Wrote generated sentence to generated-sentence-temp-1.txt.
```

Hình 2. Kết quả sinh văn bản với temperature 0.0 và 1.0.

4. Thiết lập thực nghiệm và Kết quả

Lưu ý: file test của CFIMDB trong bộ dữ liệu được cung cấp dùng nhãn placeholder -1. Vì vậy, accuracy trên CFIMDB test set chỉ được ghi nhận như kết quả chạy chương trình, không dùng làm kết luận chính về chất lượng mô hình. Các phiên bản classifier.py cần uncommand để chạy.

Các phiên bản classifier tương ứng được đặt dưới: #the best for testing … txt

```
# the best for testing cfimdb-test.txt
# import torch
# import torch.nn.functional as F
```

Phương pháp	Dataset	Hyperparameters	Dev Acc	Test Acc
Zero-shot prompting	SST-5	batch size = 10	0.213	0.224
Zero-shot prompting	CFIMDB	batch size = 10	0.502	0.213
Best tuning	SST-5	epochs = 3, lr = 1e-5, batch = 50	0.431	0.420
Best tuning	CFIMDB	epochs = 5, lr = 1e-5, batch = 10	0.894	0.496

```
In [8]: !python run_llama.py --option prompt --batch_size 10 --train data/sst-train.txt --dev data/sst-dev.txt --test data/sst-test.txt --label-names data/sst-label-mapping.json --dev_out sst-dev-prompting-output.txt --test_out sst-test-prompting-output.txt --use_gpu

args: {'train': 'data/sst-train.txt', 'dev': 'data/sst-dev.txt', 'test': 'data/sst-test.txt', 'label_names': 'data/sst-label-mapping.json', 'pretrained_model_path': 'stories42M.pt', 'max_sentence_len': None, 'seed': 1337, 'epochs': 5, 'option': 'prompt', 'use_gpu': True, 'generated_sentence_low_temp_out': 'generated-sentence-temp-0.txt', 'generated_sentence_high_temp_out': 'generated-sentence-temp-1.txt', 'dev_out': 'sst-dev-prompting-output.txt', 'test_out': 'sst-test-prompting-output.txt', 'batch_size': 10, 'hidden_dropout_prob': 0.3, 'lr': 2e-05}
load 8544 data from data/sst-train.txt
load 1101 data from data/sst-dev.txt
load 2210 data from data/sst-test.txt
eval: 100%|██████████| 111/111 [00:02<00:00, 44.42it/s]
eval: 100%|██████████| 221/221 [00:03<00:00, 65.32it/s]
dev acc :: 0.213
test acc :: 0.224
```

Hình 3. Zero-shot prompting trên SST-5.

```
In [9]: !python run_llama.py --option prompt --batch_size 10 --train data/cfimdb-train.txt --dev data/cfimdb-dev.txt --test data/cfimdb-test.txt --label-names data/cfimdb-label-mapping.json --dev_out cfimdb-dev-prompting-output.txt --test_out cfimdb-test-prompting-output.txt --use_gpu

args: {'train': 'data/cfimdb-train.txt', 'dev': 'data/cfimdb-dev.txt', 'test': 'data/cfimdb-test.txt', 'label_names': 'data/cfimdb-label-mapping.json', 'pretrained_model_path': 'stories42M.pt', 'max_sentence_len': None, 'seed': 1337, 'epochs': 5, 'option': 'prompt', 'use_gpu': True, 'generated_sentence_low_temp_out': 'generated-sentence-temp-0.txt', 'generated_sentence_high_temp_out': 'generated-sentence-temp-1.txt', 'dev_out': 'cfimdb-dev-prompting-output.txt', 'test_out': 'cfimdb-test-prompting-output.txt', 'batch_size': 10, 'hidden_dropout_prob': 0.3, 'lr': 2e-05}
load 1707 data from data/cfimdb-train.txt
load 245 data from data/cfimdb-dev.txt
load 488 data from data/cfimdb-test.txt
eval: 100%|██████████| 25/25 [00:02<00:00, 9.33it/s]
eval: 100%|██████████| 49/49 [00:04<00:00, 10.41it/s]
dev acc :: 0.502
test acc :: 0.213
```

Hình 4. Zero-shot prompting trên CFIMDB.

6. Nhận xét

- Zero-shot prompting cho kết quả thấp, phù hợp với kỳ vọng vì checkpoint stories42M.pt không được huấn luyện trực tiếp cho bài toán phân loại cảm xúc.
- Fine-tuning cải thiện rõ rệt so với zero-shot. Trên CFIMDB, dev accuracy tăng từ 0.502 lên 0.857 với cấu hình chuẩn và đạt 0.894 ở cấu hình tốt nhất.
- SST-5 khó hơn do có 5 lớp nhãn. Cấu hình tốt nhất đạt dev accuracy 0.431 và test accuracy 0.420.
- Việc giảm learning rate từ 2e-5 xuống 1e-5 giúp cải thiện kết quả ở các thí nghiệm tốt nhất.

7. Kết luận

Nhóm đã hoàn thành các thành phần cốt lõi được yêu cầu của Llama2 tối giản, bao gồm attention, RMSNorm, forward/generate, RoPE, AdamW và classification head. Các kiểm thử bắt buộc đều passed. Kết quả thực nghiệm cho thấy fine-tuning giúp cải thiện đáng kể so với zero-shot prompting, đặc biệt trên CFIMDB.